PART 1

Text Processing and Exploratory Data Analysis

Git repository and Tag

Repository: https://github.com/froggyalchemist/irwa-search-engine
Tag: IRWA-2026-part-1

AI Use Declaration

We used **Claude Sonnet 5.5** to revise the draft's structure and contents in order to get feedback and improve our work.

We used **ChatGPT (Sol and Astra)** for coding assistance for the EDA part. We provided the instructions and pseudo code and used the model to obtain the final code of each part.

Git repository and Tag **1**
AI Use Declaration **1**
Part 1: Preprocessing the documents **2**
Step 2. Data quality checks and converting data types 2
Step 3. Text processing 3
Justification 4
Part 2: EDA **5**
1. Category balance: categories and subcategories 5
2. Most frequent subcategories 6
3. Least frequent subcategories 7
4. Price distributions 8
5. Rating and discount distributions 9
6. Top brands by product quantity 9
7. Top sellers by product quantity 10
8. Stock availability 11
9. Length distributions 12
10. Most Common Terms 12
11. Named Entity Recognition 14

Part 1: Preprocessing the documents

We decided to use pandas to preprocess the documents, as is the library we are most familiar with. Our workflow has four main steps:

1. Read the data stored in the json file into a DataFrame
2. Clean the data: inspect missing values and convert data types
3. Process the text by stemming, tokenizing, etc.
4. Save the results to csv files

The sections below explain steps 2 and 3 in more detail and show the reasoning behind our choices in each step.

Step 2. Data quality checks and converting data types

After loading the data, we verify that pid exists and is unique for every product. The evaluation labels refer to products by pid, so it must identify each product without ambiguity. We then count the missing values in each field, treating empty and whitespace-only strings as missing. Figure 1 shows the result.

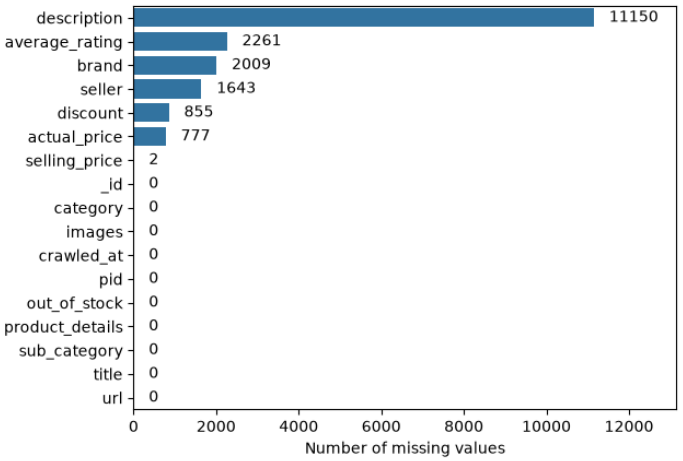


Figure 1. Number of missing values on each field.

We weren't explicitly asked to do this, but it gives us important context about the fashion products dataset — a large percentage of products have no description (40%), average rating (8%), brand (7%), or seller (5%). However, all products have a pid, title, category, and subcategory.

Next, we convert all numeric columns (prices, discounts, and ratings) from strings to floats. The column out_of_stock is already boolean, so we keep it unchanged. This step

is indispensable for multiple reasons. First, we need numeric values for the EDA later on — we cannot, for instance, find the average price of all products if they are stored as a string of text. Second, we'd lose functionality in the search engine: the user wouldn't be able to sort the search results by price, or find those with a discount.

Step 3. Text processing

In step 3, we create two tables: df and index_df.The former, df, holds all of the products' information without any text processing. The product's title, description, etc. are kept as they were originally written. For example, in the description of a product in df might read: 'Wool Black pants made out of 100% cotton'.

	pid	title	description	category	actual_price	brand	url	out_of_stock
0	TKPFCZ9EA7H5FYZH	Solid Women Multicolor Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	2999.0	York	https://www.flipkart.com/yorker-solid-men-mult...	False
1	TKPFCZ9EJZV2UVRZ	Solid Men Blue Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	1499.0	York	https://www.flipkart.com/yorker-solid-men-blue...	False
2	TKPFCZ9EHFCY5Z4Y	Solid Men Multicolor Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	2999.0	York	https://www.flipkart.com/yorker-solid-men-mult...	False
3	TKPFCZ9ESZZ7YWEF	Solid Women Multicolor Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	2999.0	York	https://www.flipkart.com/yorker-solid-men-mult...	False
4	TKPFCZ9EVXKBSUD7	Solid Women Brown, Grey Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	2999.0	York	https://www.flipkart.com/yorker-solid-men-brow...	False

Figure 2. First five rows of df. Not all fields are shown due to lack of space.

The latter, index_df,only contains the information needed to build the index. This includes the fields "title", "description", "brand", "category", "sub_category", and"seller" — things a user could type into the search bar if they wanted to find a specific piece of clothing.

Inspecting the json file, we noticed product_details contains relevant information such as a product's color, pattern or material. Therefore, we decided to include it in index_df. This field is a list of single-entry dictionaries, for example {'Fabric': 'Cotton Blend'}. We keep only the values. The keys (Fabric, Color, Pattern) repeat across many products, and they would only add noise if we kept them as terms.

	pid	title	description	brand	category	sub_category	seller	product_details
0	TKPFCZ9EA7H5FYZH	[solid, women, multicolor, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005combo2, elast, side, pocket, cotton, blen...
1	TKPFCZ9EJZV2UVRZ	[solid, men, blue, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005blue, drawstr, elast, side, pocket, cotto...
2	TKPFCZ9EHFCY5Z4Y	[solid, men, multicolor, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005combo4, elast, side, pocket, cotton, blen...
3	TKPFCZ9ESZZ7YWEF	[solid, women, multicolor, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005combo3, elast, side, pocket, cotton, blen...
4	TKPFCZ9EVXKBSUD7	[solid, women, brown, grey, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005combo1, drawstr, elast, side, pocket, cot...

Figure 3. First five rows of index_df.

All fields in index_df are pre-processed using the function build_terms(). This function performs four operations in this order:

1. Lowercase the text.
2. Tokenize with the regular expression \w+, which keeps words and numbers and drops punctuation.
3. Remove stop words using the English stop-word list from nltk. We exclude “no” and “not” from the stopword list to preserve the meaning of sentences like "shirt with no sleeves" (please give us the extra points).
4. Stem each token with the Porter stemmer.

Table 1 shows an example of how it transforms the text.

Before	Yorker trackpants made from 100% rich combed cotton giving it a rich look.Designed for Comfort,Skin friendly fabric,itch-free waistband & great for all year round use Proudly made in India
After	['yorker', 'trackpant', 'made', '100', 'rich', 'comb', 'cotton', 'give', 'rich', 'look', 'design', 'comfort', 'skin', 'friendli', 'fabric', 'itch', 'free', 'waistband', 'great', 'year', 'round', 'use', 'proudli', 'made', 'india']

Table 1. Description of the first item in the dataset before and after applying build_terms().

We assume queries will go through the same build_terms() function as the documents, otherwise stemmed document terms would not match raw query terms.

Justification

We use two dataframes to separate the data shown to the user from the data needed to build the index. df keeps the original text because stemmed terms are unreadable in a results list (friendli instead of “friendly”). index_df stores the preprocessed terms, so the next part of the project can build the index without repeating the preprocessing. It is also the table we use for the exploratory data analysis (EDA). Both tables share pid, which links every index entry to the full product information.

For the text fields, we compared two options: merging them into a single string, or keeping them as separate fields. In the end, we chose to keep them separate. The first option is the simplest, but it’s less flexible. If we merge all fields, a match in the title would count the same as a match in the seller name.

The second option would let us give each field its own weight, which we could tune with validation_labels.csv in the next part of the project. The disadvantage of separate fields is more complexity: scoring must combine multiple fields. It also makes the index larger and harder to build. However, space is not the real problem, because the dataset is small (~60 MiB in json format, ~45 MiB in csv). We accept the extra complexity of this option because we think it'll lead to better performance in the future.

Finally, we store both tables as csv. It is the natural format for a dataframe, and we can easily convert it back to json if needed. It has one limitation, though: list columns (product_details and the term lists in index_df) are written as strings, so they must be parsed correctly when the files are loaded.

Part 2: EDA

Exploratory data analysis helps us understand the composition of the product collection and identify characteristics that may affect retrieval. We examine category balance, prices, ratings, discounts, brand and seller representation, and stock availability. These analyses inform decisions about searchable fields, structured filters, ranking signals, and the interpretation of retrieval results.

1. Category balance: categories and subcategories

We can see that Clothing and Accessories contains 27,118 products, which represents 96.57% of the collection. The footwear represents 3.28%, while Bags, Wallets & Belts and Toys together account for less than 0.2%. The dataset has clearly more clothing pieces than any other type of product.

With an Information Retrieval perspective, we believe that this imbalance will make terms associated with clothing occur across much of the collection and consequently offer less discrimination power between products. We will probably need more specific information, such as garment type, material, fit, and color to distinguish relevant items.

Additionally, our corpus does not have many candidates for queries looking for minority categories. In this sense, for those queries we can expect poorer results, but which will probably be due to limited corpus coverage rather than only an indexing or ranking problem.

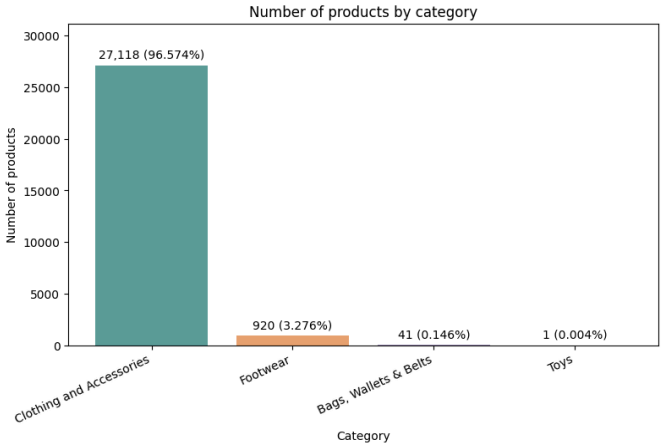


Figure 4. Number of products by category.

2. Most frequent subcategories

Topwear contains 15,310 products (54.52%) of the entire collection. Bottomwear and Winter Wear contain 3,666 and 2,493 products, respectively. Together, these three subcategories represent the 76.46% of all products. The concentration within these groups means that matching a general product type can still leave many candidates. Inside Topwear, for example, identifying a shirt is insufficient to distinguish its intended gender, fabric, sleeve type, or color.

This supports including `product_details` together with titles and descriptions in the searchable information. It also shows us the importance of field weights: matches in a title or relevant product attribute may provide stronger evidence than matches in generic descriptive text. We will have to test the effectiveness of these weights.

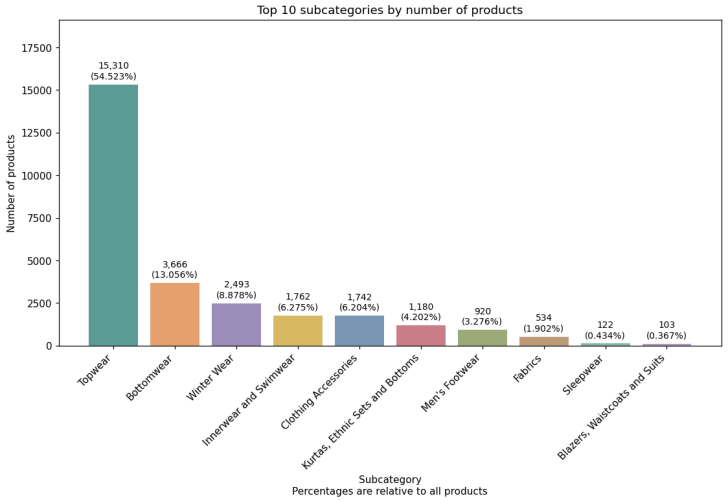


Figure 5. Top 10 subcategories by number of products.

3. Least frequent subcategories

We can see that several subcategories contain only one product, while others shown in the Bottom 5 contain four. This reveals a long tail of product types with very limited representation.

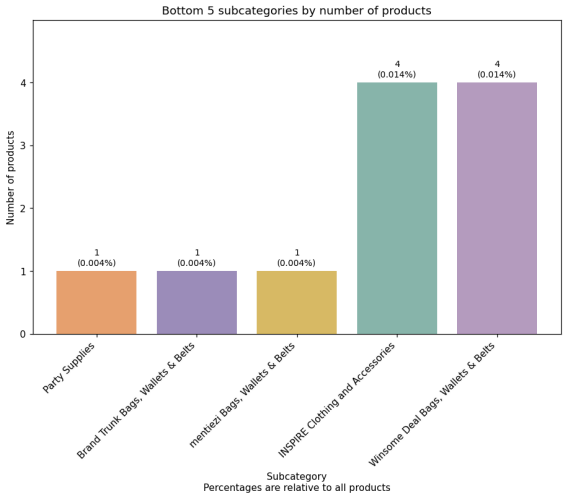


Figure 6. Bottom 5 subcategories by number of products.

For these groups, adding constraints to a query can quickly reduce an already small candidate set to zero. Therefore, a lack of results does not necessarily mean that the preprocessing failed; the collection may simply contain no product satisfying all requested attributes. Examining candidate counts will help distinguish coverage limitations from retrieval errors.

Some low-frequency subcategory labels also contain apparent brand names. This suggests that the category vocabulary may not be well defined. Consequently, subcategories can support navigation and filtering, but they should not be treated as complete, verified product information. We preserve the original labels rather than correct ourselves those categories.

4. Price distributions

Both selling price and `actual_price` have right-skewed distributions: most products occupy lower price ranges, with a smaller number extending into a high-price tail. Their medians are 545 and 1,199, respectively. We use medians to describe typical prices because high values can disproportionately influence means.

For IR, price is more useful as a structured attribute than as an ordinary textual term. A query like “shirts under 1,000” requires a numerical comparison; matching the token `1000` would not retrieve all products below that threshold. Our numerical conversion therefore prepares these fields for price-range filtering and price-based sorting.

Price should remain separate from textual relevance. An inexpensive product is not necessarily relevant to a query, and an expensive product is not necessarily irrelevant. A suitable future design would identify products matching the requested characteristics before applying an explicit price constraint or sorting preference.

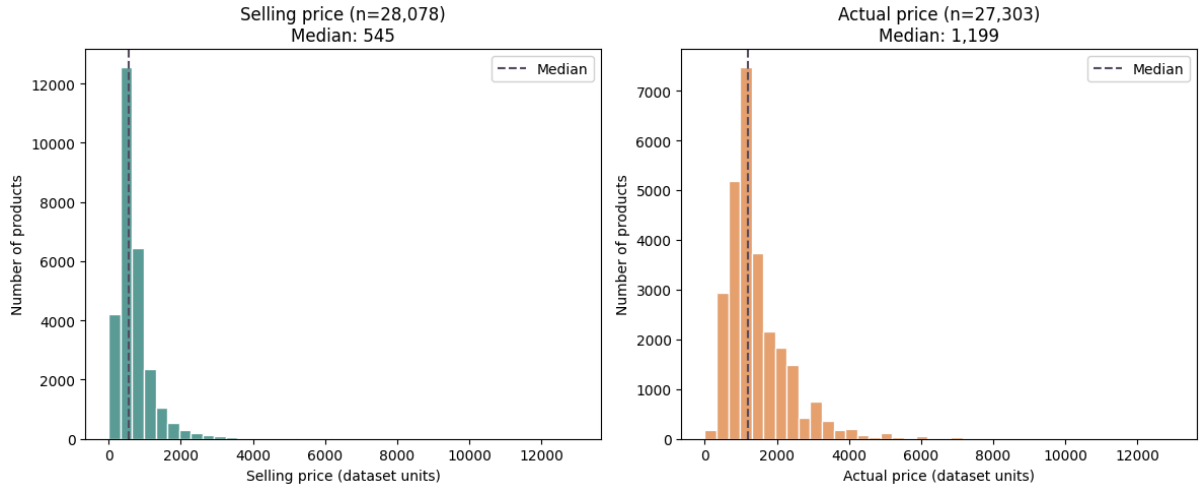


Figure 7. Distribution of selling price and actual price.

5. Rating and discount distributions

Ratings are concentrated towards the higher end of the scale, with a median of 3.8. Discounts also tend to be substantial, with a median of 53%. These attributes distinguish products beyond their textual descriptions, but they represent different information from query relevance.

A high rating does not establish that a product satisfies the requested color, material, or garment type. Rating could instead provide a secondary ranking signal among products that already match the query, or support an explicit condition such as “rated at least four stars.”

Similarly, discount can support requests for discounted products or a minimum discount threshold. It should not automatically dominate ranking: a heavily discounted

item may still be unrelated to the query, and a large percentage reduction does not necessarily imply a low selling price.

We leave missing ratings and discounts as unknown rather than fill them with zero or the mean. Like this, we preserve their distributions and avoids assigning unsupported quality or promotional signals to products. If these attributes are later incorporated into ranking, missing values will require an explicit policy and evaluation.

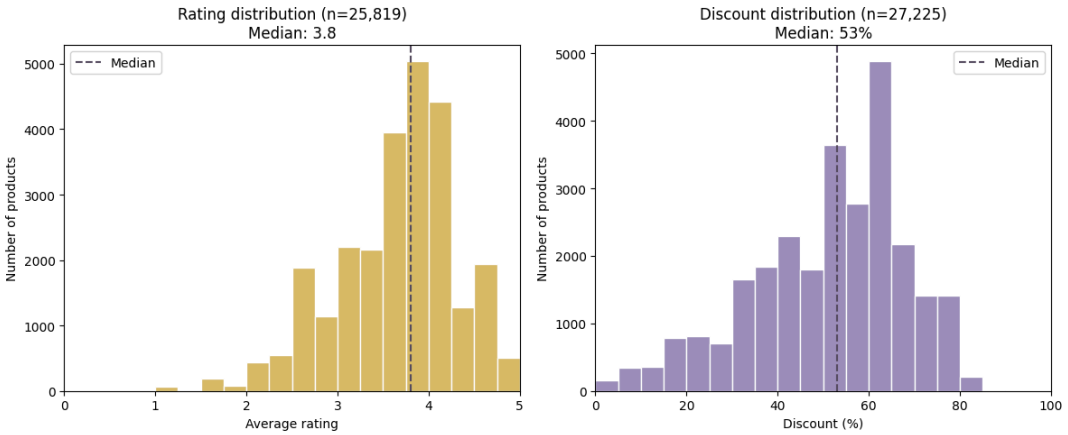


Figure 8. Distribution of average product rating and discount.

6. Top brands by product quantity

ECKO Unl has the largest representation, with 951 products, followed by Free Authori with 860 and ARBO with 806. However, 2,009 products have no recorded brand. Brand information can be useful when the query explicitly names a brand. Preserving it as a separate field allows an exact brand condition or a brand-specific relevance contribution without confusing the brand with words appearing elsewhere in the document.

Brand representation alone should not determine ranking for queries that do not express a brand preference. Otherwise, products could receive an advantage unrelated to the user's requirements. Missing brands should likewise not exclude products from general searches: these records still contain potentially relevant titles and attributes.

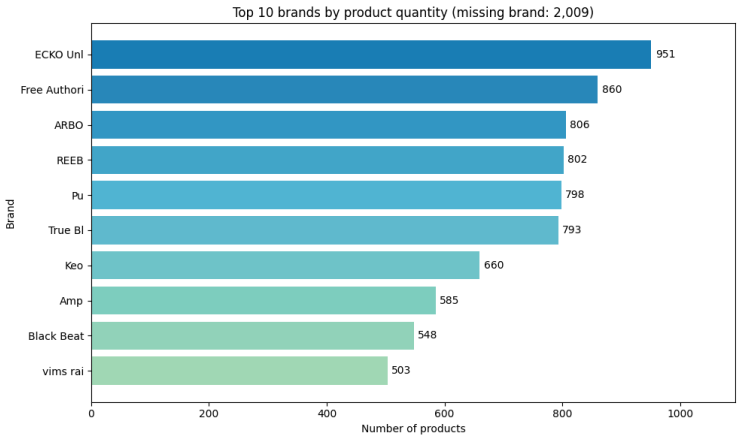


Figure 9. Top 10 brands by product quantity.

7. Top sellers by product quantity

RetailNet contributes with 1,411 products, followed by SandSMarketing with 887 and BioworldMerchandising with 842. Seller information is missing for 1,643 products.

For attribute-based queries such as “men slim jeans blue,” seller identity usually provides weaker evidence of relevance than product type, fit, and color. Giving seller terms the same importance as title or attribute matches could therefore introduce irrelevant matching evidence.

Keeping seller information separate allows it to receive limited influence in general product searches while remaining available for queries that explicitly specify a seller. The observed counts describe representation, so they do not justify treating frequently occurring sellers as more relevant or trustworthy.

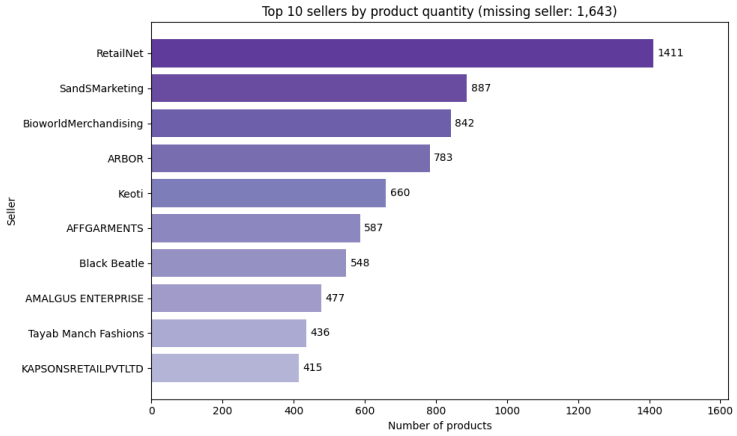


Figure 10. Top 10 sellers by product quantity.

8. Stock availability

The collection contains 26,436 in-stock products (94.15%) and 1,644 out-of-stock products, representing 5.85%.

Stock availability affects the usefulness of a shopping result even when its description matches the query. A relevant but unavailable product may not satisfy a user who wants to purchase immediately. The boolean stock field can therefore support an explicit availability filter or an optional preference for available products.

We retain out-of-stock products in the collection because availability is distinct from topical relevance. This preserves information for evaluation and allows the eventual search interface to control whether unavailable items are displayed. The recorded status represents availability when the data was collected, rather than a live inventory guarantee.

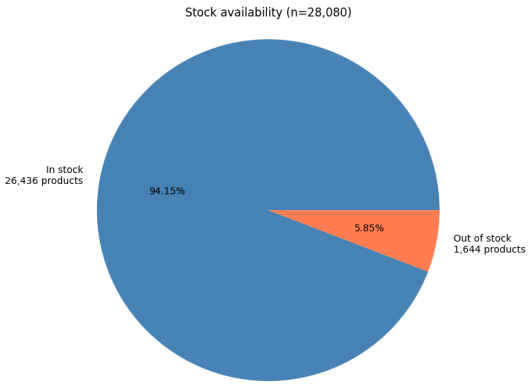


Figure 11. Percentage of items in stock and out of stock.

9. Length distributions

We decided to measure length using the token lists we created in the preprocessing phase. The counts we will discuss here refer to the processed tokens rather than the original words or characters. Regarding the final token count, we sum the seven indexed fields and preserve tokens that are repeated, we do not merge the fields inside the index.

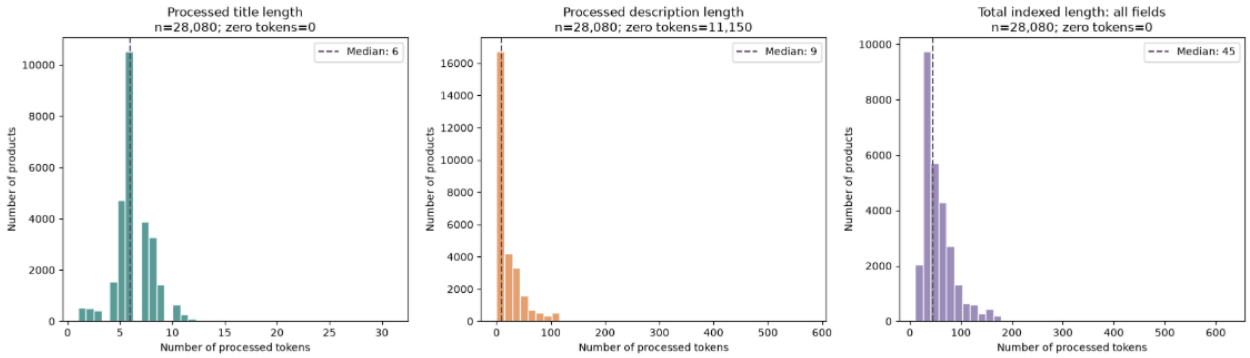


Figure 12. Distribution of products' title, description, and total lengths.

We can observe that titles are mostly short and with a consistent length overall. The median length is 6 processed tokens per title, while 95% of them contain at most 9 tokens.

In contrast, descriptions vary much more and some are even 579 tokens long. However, 11,150 products, representing 39.7%, have no description. It follows from this that descriptions provide a limited coverage of our product catalogue: titles and other indexed fields will be still important to represent these products.

Taking into account the total indexed length, i.e. all indexed fields, the median is 45 tokens. Observing this variation in text length will make us consider length normalization when we design a retrieval stage.

10. Most Common Terms

We have thought to compute frequencies separately for titles and descriptions. In total occurrences we count every single appearance of a term, and document frequency we count each product containing that term only one time. For the percentages, we have used all 28,080 products as the denominator, including those that have no description.

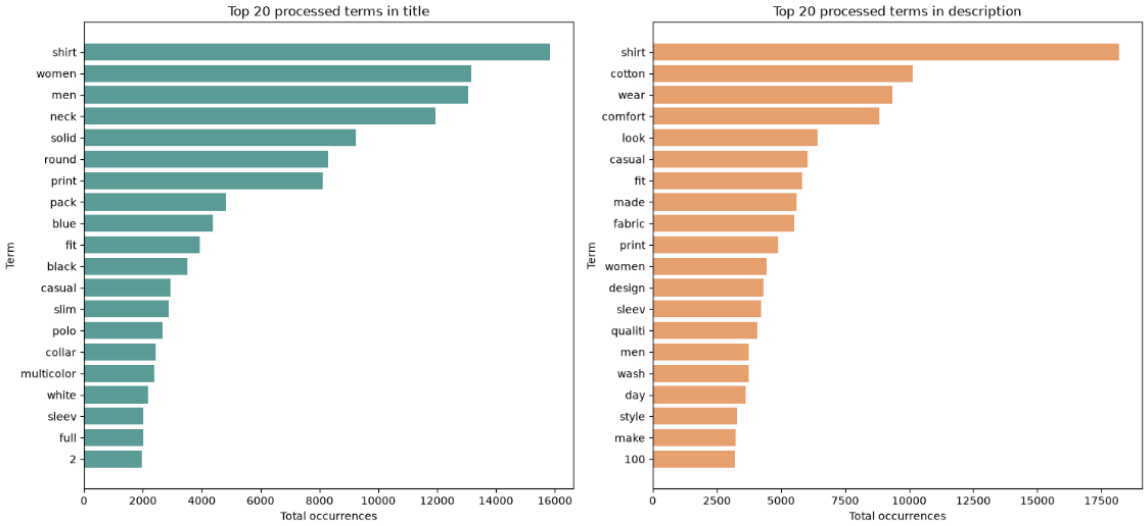


Figure 13. Top 20 processed terms in titles and descriptions

In titles, the most frequent terms describe product types (shirt, polo), the audiences (men, women), and attributes (neck, solid). The descriptions add material information (cotton) together with vocabulary that can be considered more promotional (comfort, look) or about how to treat the product (wash).

	term	corpus_count	document_count	products_pct
0	shirt	15838	15838	56.40
1	women	13146	13041	46.44
2	men	13060	13021	46.37
3	neck	11952	11952	42.56
4	solid	9246	9215	32.82
5	round	8301	8301	29.56
6	print	8123	8106	28.87
7	pack	4839	4787	17.05
8	blue	4378	4282	15.25
9	fit	3923	3923	13.97
10	black	3503	3501	12.47
11	casual	2927	2927	10.42
12	slim	2893	2828	10.07
13	polo	2668	2668	9.50
14	collar	2427	2360	8.40
15	multicolor	2373	2370	8.44
16	white	2175	2168	7.72
17	sleev	2026	2026	7.22
18	full	2011	2011	7.16
19	2	1964	1958	6.97

	term	corpus_count	document_count	products_pct
0	shirt	18180	8362	29.78
1	cotton	10139	7248	25.81
2	wear	9353	5416	19.29
3	comfort	8836	6796	24.20
4	look	6413	4398	15.66
5	casual	6034	4824	17.18
6	fit	5831	4602	16.39
7	made	5619	4399	15.67
8	fabric	5516	4340	15.46
9	print	4888	3294	11.73
10	women	4424	3396	12.09
11	design	4301	3605	12.84
12	sleev	4221	3330	11.86
13	qualiti	4087	2849	10.15
14	men	3736	3029	10.79
15	wash	3731	2142	7.63
16	day	3617	3283	11.69
17	style	3275	2546	9.07
18	make	3233	2460	8.76
19	100	3210	2936	10.46

Figure 14. Top most frequent terms with corpus_count, document_count and percentages

If we have a look at total occurrences and document frequency, we can see different patterns. An example of this is shirt, which occurs 18,180 times in descriptions, but in

a total of 8,362 products. This helps us detect terms that are widely repeated within individual products.

We can also observe that of the 16,930 nonblank product descriptions, only 5,683 are distinct. In this sense, word frequencies can also be affected by the same text being used in multiple products.

We provide also a word cloud visualization of the most frequent terms in titles and descriptions:



Figure 15. Word clouds of most frequent processed terms in titles and descriptions

11. Named Entity Recognition

As suggested in the project guidelines, we perform a brief Named Entity Recognition exploration. We have decided to apply it to original descriptions in order to preserve the original wording before tokenization and stemming.

We use [spaCy's en_core_web_sm model] and exclude empty descriptions. We process each description once to avoid repeated computation. We associate the predictions with every product and show their description. The model has produced 48,142 predicted mentions in 14,047 products. We inspect individual predictions and summarize each category with its total mentions and some frequent examples.

label	mentions	examples (mention counts)
ORG	14333	keoti (431); printed (382); the amazing comfort of a shoefly polo t-shirts (247)
PERSON	9681	arbour (431); casual wear (418); track pant (400)
PERCENT	5483	100% (3005); 60% (1095); 40% (1048)
DATE	5425	daily (543); weekend (541); some days (529)
GPE	4157	cotton (918); kurta (911); india (624)
CARDINAL	3548	half (732); one (423); 2 (354)
NORP	1792	jeans (582); joggers (237); indian (203)
WORK_OF_ART	1715	shirt for women women (192); t shirt for men women (185); comfort fit (93)
PRODUCT	516	round (130); everland (48); t-shirts (46)
QUANTITY	311	30.5 inches (83); 2-inch (40); one-inch (21)
TIME	309	winter (60); long hours (56); daytime (40)
LAW	189	stretcahble with dri fit comfortable fit (137); this parti wear (16); anti-slip silicon (pack of 3 pairs (14)
ORDINAL	184	first (139); second (26); 1pc (15)
FAC	173	polo (39); axmann (34); scott international (18)
LOC	112	vest (33); south east asia (28); pooja (8)
EVENT	94	traditional indian wear kurta (55); jacket (22); cold (3)
MONEY	73	46 inches / length (67); 1778 cm (4); 44 inches (2)
LANGUAGE	47	mandarin (46); english (1)

Figure 16. Top labels for named entity recognition

If we observe the most frequent predictions, we can see a lot of labeling problems. For example, “casual wear” and “track pant” are classified as people, “cotton” and “kurta” as countries or cities and “jeans” and “joggers” as nationalities or religious/political groups.

Some other predictions are plausible, including percentages, measurements, and “India” as a geographical entity. However, these cases are not enough to evaluate this simple NER as a reliable tool. We might explore different tools, but for now we believe we will not use it to enrich the search index.